Fiche de révision – Soutenance TP DevOps

MEGNE · ING2 – IAI Libreville · Projet `system_metrics_agent` · 15 octobre 2026

> **J'ai automatisé tout le chemin du code jusqu'à l'image publiée : chaque modification est testée, analysée (SonarQube), scannée (Trivy) puis publiée sur Docker Hub, sans intervention manuelle.**

1. Objectif du TP

Le but est d'appliquer la démarche **DevOps** : livrer une application vite, souvent et sans erreur, grâce à l'automatisation.

Image simple : une **usine**. Le développeur dépose son code à l'entrée. Une chaîne automatique le contrôle (tests, qualité, sécurité). Elle l'emballe dans une boîte standard (image Docker) puis l'envoie à l'entrepôt (Docker Hub). Si un contrôle échoue, la chaîne s'arrête et rien de défectueux ne sort.

Concrètement, le TP demandait de :

1. Forker le projet `Mficius/system_metrics_agent` et travailler avec Git (branches, pull requests).
2. Écrire les **Dockerfiles** de l'API et de l'agent.
3. Lancer les deux ensemble avec **Docker Compose**.
4. Créer un pipeline **CI/CD GitHub Actions** : tests, SonarQube avec Quality Gate, scan Trivy, push sur Docker Hub.
5. Démontrer qu'un échec bloque bien le pipeline.
6. Rédiger un rapport (README).

2. Le projet de départ

`system_metrics_agent` est une petite application Python en deux parties qui se parlent par HTTP.

Composant	Rôle	Technologies
Agent	Collecte CPU, RAM et charge système toutes les N secondes, puis les envoie à l'API	`psutil`, `subprocess` (commande `uptime`), `requests`, `.env`
API	Reçoit et stocke les métriques, les rend consultables	FastAPI, Uvicorn

Routes de l'API : `GET /health`, `POST /metrics`, `GET /metrics`, `GET /metrics/latest`.

3. Qu'est-ce qu'une image Docker ?

Définition à retenir : une image Docker est un **modèle figé (en lecture seule)** qui contient tout ce qu'il faut pour faire tourner une application : un système de base, l'interpréteur (Python), les bibliothèques et le code. On ne l'exécute pas directement : on lance à partir d'elle un ou plusieurs **conteneurs**.

Analogie : l'image est le **moule** (ou la recette), le conteneur est le **gâteau** sorti du moule. Avec un seul moule, on peut faire autant de gâteaux identiques qu'on veut.

Dockerfile (recette) **→ docker build →** Image (moule) **→ docker push →** Docker Hub (entrepôt) **→ docker run →** Conteneur(s) (gâteaux)

Ce qu'il faut savoir dire :

- **Elle se construit à partir d'un Dockerfile** avec `docker build`. Chaque instruction (`FROM`, `COPY`, `RUN`…) ajoute une **couche**.
- **Elle est faite de couches empilées.** Docker réutilise les couches déjà construites (cache), ce qui accélère les builds. Deux images qui partagent la même base ne la stockent qu'une fois.
- **Elle est identifiée par un nom et un tag** : `nom:tag`. Exemple : `douguyblaise4747/metrics-api:latest`. Le tag sert de version ; dans mon pipeline, chaque image reçoit aussi le SHA du commit comme tag.
- **Elle est immuable.** Pour changer le code, on reconstruit une nouvelle image ; on ne modifie pas l'ancienne.
- **Elle est stockée et partagée dans un registre**, comme Docker Hub (`docker push` pour envoyer, `docker pull` pour récupérer).
- **Elle est portable** : la même image tourne de la même façon sur mon PC, sur le runner GitHub ou sur un serveur. Fini le « ça marche chez moi ».

	Image	Conteneur
Nature	Modèle figé, en lecture seule	Instance en cours d'exécution
Création	`docker build` (depuis un Dockerfile)	`docker run` ou `docker compose up` (depuis une image)
Nombre	Une seule par version	Autant qu'on veut à partir d'une image
Liste	`docker images`	`docker ps`

Dans mon TP : deux images, `metrics-api` (252 Mo sur disque, 59,7 Mo compressée) et `metrics-agent` (203 Mo, 49,1 Mo compressée), construites sur la base `python:3.12.15-slim-trixie` (179 Mo).

4. Les outils utilisés

Outil	Définition simple	Ce qu'il fait dans mon TP
Git	Logiciel qui garde l'historique de chaque modification du code (commits)	Branches `feature/devops` et `feature/demo-echec`, `git revert`
GitHub	Site qui héberge les dépôts Git en ligne	Fork, pull requests #1 et #2, exécution du pipeline
Fork	Copie personnelle du dépôt de quelqu'un d'autre	Copie de `Mficius/system_metrics_agent` sur mon compte

Outil	Définition simple	Ce qu'il fait dans mon TP
Pull request	Demande de fusion d'une branche dans `main`, avec relecture et tests	Le pipeline tourne sur chaque PR avant la fusion
pytest	Outil qui lance les tests automatiques Python	22 tests, 92 % de couverture
black / flake8	black vérifie la mise en forme ; flake8 détecte les erreurs de style	Bloquent le pipeline si le code est mal écrit
Docker	Outil qui emballe une application et ses dépendances pour qu'elle tourne partout pareil	Images de l'API et de l'agent
Conteneur	Application qui tourne, isolée, à partir d'une image. Plus léger qu'une VM : il partage le noyau de la machine au lieu d'embarquer un système complet	Un conteneur `api` et un conteneur `agent`
Dockerfile	Fichier texte = la recette qui décrit comment construire l'image, étape par étape	`Dockerfile.api` et `Dockerfile.agent`
Docker Compose	Outil qui lance plusieurs conteneurs ensemble à partir d'un seul fichier YAML	`docker-compose.yml` : api + agent sur le réseau `metrics-net`
Docker Hub	Registre en ligne où l'on stocke et partage des images	`douguyblaise4747/metrics-api` et `metrics-agent`
CI/CD	CI = intégration continue (tester à chaque modification). CD = livraison continue (publier automatiquement)	Tout le chaînage test → analyse → scan → push
GitHub Actions	Service de GitHub qui exécute le pipeline décrit dans un fichier YAML	`.github/workflows/ci-cd.yml`
SonarQube (Cloud)	Outil d'analyse de la qualité et de la sécurité du code	A détecté une injection dans les logs, mesure la couverture
Quality Gate	Liste de seuils à respecter ; si un seuil n'est pas atteint, le pipeline échoue	« Sonar way » : notes A, couverture ≥ 80 %, duplications ≤ 3 %
Trivy	Scanner qui cherche les vulnérabilités connues (CVE) dans une image	Échec si une faille HIGH ou CRITICAL corrigeable est trouvée
Secrets GitHub	Coffre-fort chiffré pour les mots de passe et tokens	`SONAR_TOKEN`, `SONAR_HOST_URL`, `DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`
Token	Mot de passe spécial, limité et révocable, donné à un programme	Token SonarCloud et token Docker Hub (Read & Write)
pip-tools (pip-compile)	Génère la liste exacte des dépendances avec leurs empreintes SHA-256	Fichiers `requirements-*.txt` verrouillés
WSL2	Linux intégré à Windows	Nécessaire pour faire tourner Docker Desktop

Vocabulaire GitHub Actions :

- **Workflow** : le pipeline complet (le fichier YAML).
- **Job** : un groupe d'étapes ; ici `test` et `build-scan-push`.
- **Step** : une étape (une commande ou une action).
- **Runner** : la machine virtuelle de GitHub qui exécute le job.
- **needs** : un job attend la réussite d'un autre.

- **Matrice** : le même job lancé pour plusieurs valeurs (`api` et `agent`).

5. Ce que j'ai réalisé

Toutes les parties du TP sont faites et vérifiées : le pipeline est vert sur `main` et les images sont sur Docker Hub.

Git et GitHub. Fork du projet, travail sur des branches, fusion par pull requests (PR #1 `feature/devops`, PR #2 `feature/demo-echec`).

Code et tests. Correction des tests qui échouaient sous Windows, ajout de tests pour l'API, la config et l'agent : 22 tests, couverture passée de 58 % à 92 %. Correction de la faille signalée par SonarQube dans `api.py`.

Dockerfiles. Bonnes pratiques appliquées :

- **multi-stage** : une étape construit, l'image finale ne garde que le nécessaire (plus petite) ;
- image de base avec un **tag précis** (`python:3.12.15-slim-trixie`), pas `latest` ;
- **utilisateur non-root** (`app`, UID 10001) : si l'appli est piratée, l'attaquant n'est pas administrateur ;
- **HEALTHCHECK** sur `/health` pour savoir si l'API répond ;
- `.dockerignore` pour ne pas copier `.git`, `.env`, les tests… ;
- dépendances **verrouillées avec empreintes** (`--require-hashes`) ;
- paquet `procps` ajouté à l'agent pour avoir la commande `uptime`.

Docker Compose. Un seul `docker compose up --build` lance l'API et l'agent sur un réseau dédié. L'agent joint l'API par son nom (`http://api:8000/metrics`) et attend qu'elle soit saine (`depends_on: condition: service_healthy`). Redémarrage automatique (`restart: unless-stopped`).

Pipeline CI/CD, à chaque push ou pull request :

push / PR → black + flake8 → pytest + couverture → SonarQube + Quality Gate → build image → scan Trivy → push Docker Hub (main seulement)

1. **Job `test`** : installation des dépendances verrouillées → `black` et `flake8` → `pytest` avec couverture → analyse SonarQube → attente de la Quality Gate. Si un point échoue, tout s'arrête.
2. **Job `build-scan-push`** (seulement si `test` réussit, une fois pour `api` et une fois pour `agent`) : construction de l'image → scan Trivy → connexion à Docker Hub → push avec les tags `<sha>` et `latest`.
3. Le push n'a lieu **que sur `main`**, jamais sur une pull request.

Sécurité du pipeline. Secrets dans GitHub (jamais dans le code), actions épinglées par SHA, droits limités (`contents: read`), tokens exposés révoqués et remplacés.

Démonstration d'échec. Sur `feature/demo-echec`, un test volontairement faux (`"ko"` au lieu de `"ok"`) a fait échouer le job `test` : build et push ignorés, aucune image publiée. Un `git revert` a remis le pipeline au vert.

Rapport. README réécrit : architecture, installation, Compose, endpoints, CI/CD, choix techniques, tailles d'images, difficultés.

6. Difficultés rencontrées

Problème	Cause	Solution
Tests du collecteur en échec sous Windows	`platform.system()` n'était pas simulé	Ajout d'un `@patch` dans les tests

Problème	Cause	Solution
`uptime` introuvable dans le conteneur agent	L'image slim ne contient pas `procps`	Installation de `procps` dans `Dockerfile.agent`
Docker Desktop ne démarrait pas	WSL2 absent	`wsl --install --no-distribution` puis redémarrage
Workflow YAML invalide	`--only-binary :all:` : deux-points suivis d'un espace	Commande mise dans un bloc `run: |`
Pipeline pas lancé sur le fork	GitHub désactive Actions par défaut sur un fork	Activation dans l'onglet Actions
Couverture insuffisante pour la Quality Gate	58 % alors que 80 % sont exigés	Nouveaux tests : 92 %
Injection dans les logs (SonarQube)	Le champ `agent` envoyé par le client était écrit tel quel dans les logs	Ligne de log réécrite sans ce champ
« Not authorized » SonarCloud, « unauthorized » Docker Hub	Token supprimé ou mal copié ; token Docker Hub en lecture seule	Nouveaux tokens (Docker Hub en Read & Write) dans les secrets
Tokens visibles par erreur	Captures d'écran, collage dans le terminal	Révocation immédiate et remplacement
Action Trivy compromise (mars 2026)	Attaque de la chaîne d'approvisionnement	Version saine v0.35.0 épinglée par son SHA

7. Questions probables du jury

Quelle différence entre une image et un conteneur ?
L'image est le modèle figé ; le conteneur est une instance qui tourne à partir de cette image. Une image peut donner plusieurs conteneurs.

Quelle différence entre un conteneur et une machine virtuelle ?
Une VM embarque un système d'exploitation complet. Un conteneur partage le noyau de la machine hôte : il est plus léger et démarre en quelques secondes.

Pourquoi un build multi-stage ?
Pour que l'image finale ne contienne que le nécessaire à l'exécution : elle est plus petite et expose moins de failles.

Pourquoi un utilisateur non-root ?
Si l'application est compromise, l'attaquant n'a pas les droits administrateur dans le conteneur.

Pourquoi Docker Compose plutôt que deux `docker run` ?
Un seul fichier décrit tout (réseau, variables, ordre de démarrage) et une seule commande lance l'ensemble.

Comment l'agent trouve-t-il l'API ?
Compose crée un réseau où chaque service est joignable par son nom : `http://api:8000`.

Que se passe-t-il si un test échoue ?
Le job `test` échoue, le job `build-scan-push` ne démarre pas (`needs: test`), aucune image n'est publiée. Je l'ai démontré avec la branche `feature/demo-echec`.

Qu'est-ce que la Quality Gate ?
Un ensemble de seuils de qualité (notes A, couverture ≥ 80 %, duplications ≤ 3 %). Avec `sonar.qualitygate.wait=true`, le pipeline échoue si elle n'est pas validée.

Différence entre SonarQube et Trivy ?
SonarQube analyse **mon code source** (bugs, failles, couverture). Trivy analyse **l'image construite** (failles connues dans les paquets et bibliothèques).

Pourquoi ne pas pousser l'image sur une pull request ?
Une PR n'est pas encore validée ; seul le code fusionné dans `main` doit être publié.

Où sont les mots de passe ?
Dans les secrets chiffrés de GitHub, jamais dans le code. Ce sont des tokens limités et révocables.

Pourquoi l'agent dans Docker voit 16 Go de RAM et pas 32 Go ?
Il mesure la VM WSL2 qui fait tourner Docker, pas le PC. Pour superviser un vrai serveur, il faut l'installer sur l'hôte ou lui donner accès aux informations de l'hôte.

8. Pitch d'une minute

> *« Mon TP consistait à mettre en place une chaîne DevOps complète pour une application Python de supervision : un agent qui collecte le CPU, la RAM et la charge système, et une API FastAPI qui reçoit ces métriques.*
>
> *J'ai d'abord conteneurisé les deux composants avec des Dockerfiles multi-stage, un utilisateur non-root et un healthcheck. Une image Docker, c'est le modèle figé qui contient l'application et tout ce dont elle a besoin ; les conteneurs sont lancés à partir d'elle. Docker Compose démarre les deux conteneurs ensemble sur un réseau dédié.*
>
> *Ensuite, j'ai créé un pipeline GitHub Actions. À chaque modification, il vérifie le style, lance 22 tests avec 92 % de couverture, analyse le code avec SonarQube et sa Quality Gate, construit les images, les scanne avec Trivy et, seulement sur `main`, les publie sur Docker Hub.*
>
> *J'ai prouvé que le pipeline bloque bien les erreurs avec un test volontairement faux : aucune image n'a été publiée. Enfin, j'ai sécurisé le tout avec des secrets GitHub et des actions épinglées. »*